Master 2 TNAH, École nationale des chartes
10 novembre 2026

Marion Charpier



Introduction au langage de programmation Python

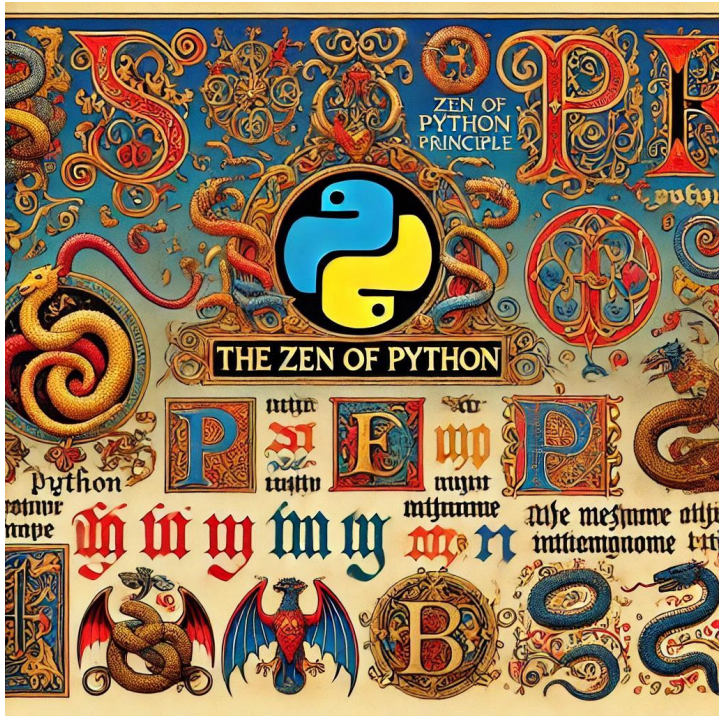
Du tableur à Python : CSV

Programme du cours

Séance	Date	Sujet
1	29 septembre 2026	Les bases de la programmation
2	6 octobre 2026	Créer ses propres outils : fonctions et modules
3	14 octobre 2026	Programmation orientée objet 1 : créer ses propres objets
4	20 octobre 2026	Programmation orientée objet 2 : héritage et décorateurs
5	27 octobre 2026	Lire et enregistrer des données : JSON
6	4 novembre 2026	Explorer les collections numérisées : IIIF
7	10 novembre 2026	Du tableur à Python : CSV

Plan de la séance 7

1) **Correction de l'interrogation**
2) **Le CSV : un tableur en texte brut**
   a) Lire un CSV dans un éditeur de texte
   b) Les pièges du CSV : séparateur et virgule décimale
3) **La bibliothèque pandas**
   a) Une question, deux réponses : la boucle ou une ligne
   b) DataFrame et Series : des objets avec attributs et méthodes
4) **Lire et explorer un tableau**
   a) Lire un fichier : read_csv()
   b) Explorer : head(), tail(), info(), describe(), columns et shape
5) **Sélectionner et filtrer**
   a) Sélectionner une ou plusieurs colonnes
   b) Sélectionner des lignes : loc et iloc
   c) Filtrer avec une condition
6) **Nettoyer les données**
   a) Repérer et traiter les valeurs manquantes : isna(), dropna(), fillna()
   b) Créer une nouvelle colonne : siecle (avec astype())
   c) Trier et compter : sort_values() et value_counts()
7) **Enregistrer un CSV : to_csv()**
8) **Notions à retenir**



D’où on part

Nos données jusqu’ici

Depuis la séance 1, le corpus est une liste de dictionnaires :

```
corpus = [
    {"cote": "MS 24",
     "bibliotheque": "Aberdeen, University Library",
     "date": 1200, "numerise": True},
    {"cote": "MS. Ashmole 1511",
     "bibliotheque": "Oxford, Bodleian Library",
     "date": 1210, "numerise": True},
]
```

En séance 5, on l’a enregistré en JSON. En séance 6, on a lu les manifestes publiés par les bibliothèques.

La question

Comment ouvrir ce corpus dans un tableur ? Comment le trier, le filtrer, puis le partager avec un collègue qui ne code pas ?

La réponse

- **Le CSV** : un fichier texte que lisent aussi bien un tableur que Python.
- **pandas** : la bibliothèque Python qui manipule ces tableaux, comme un tableur, mais avec du code.

Lire un CSV dans un éditeur de texte

corpus.csv ouvert dans un éditeur de texte (VS Code, Sublime Text, TextEdit), et non dans un tableur :

```
sigle;version;ville;bibliotheque;cote;date;folios;commentaire;manifeste
C;[*C1];Bern;Burgerbibliothek;318;9;7–22v;;https://www.e-codices.unifr.ch/metadata/iiif/bbb-0318/manifest.json
B-Is;[*B-Is1];Città del Vaticano;BAV;Pal. lat. 1074;10–11;1–22;;https://digi.vatlib.it/iiif/MSS_Pal.lat.1074/manifest.json
Sous-version H-B-Is;[*B-Is3];London;BL;Stowe 1067;12 (inc.);1–15v;;https://bl.digirati.io/iiif/ark:/81055/vdc_100056068493.0x000001
2F;[*B-Bes6];Aberdeen;UL;24;12 (ex.);1–103;;
2F;;Cambridge;Corpus Christi College;53;14 (inc.);189–210;;https://dms-data.stanford.edu/data/manifests/Parker/gs233db8425/manifest.json
2F;[*B-Bes1];London;BL;Add. 11283;12 (inc.);1–41;;https://bl.digirati.io/iiif/ark:/81055/vdc_100055965341.0x000001
2F;;London;BL;Harley 4751;12 (ex.);1–74v;;https://bl.digirati.io/iiif/ark:/81055/vdc_100056056799.0x000001
2F;;Oxford;Bodl. Lib.;Bodl. 533;13;1–29v;;https://iiif.bodleian.ox.ac.uk/iiif/manifest/36652b63-d9b1-4480-a88b-769d558bf238.json
2F;;Oxford;Bodl. Lib.;Ashmole 1511;12 (ex.);1–104;;https://iiif.bodleian.ox.ac.uk/iiif/manifest/faeff7fb-f8a7-44b5-95ed-cff9a9ffd198.jso
2F;;Oxford;Saint John's College;61;13;1–103;;https://iiif.bodleian.ox.ac.uk/iiif/manifest/3c6c4b0c-67e4-4cc2-aa2e-e5f3464944e5.json
DC;;Paris;BNF;lat. 2843E;13 (ex.);64–79v;DC ?;https://gallica.bnf.fr/iiif/ark:/12148/btv1b104647695/manifest.json
```

Comment le lire

- **La première ligne est l'en-tête** : elle donne le nom des colonnes.
- **Chaque ligne suivante est une notice** : un manuscrit par ligne.
- **Le séparateur sépare les cellules** : ici, le point-virgule.
- **Deux séparateurs qui se suivent (;;) signalent une cellule vide** : Ashmole 1511 n'a pas de version, Aberdeen n'a pas de manifeste.

Une ligne, un dictionnaire

Les noms de colonnes jouent le rôle des clés :

{"sigle": "2F", "ville": "Aberdeen", "bibliotheque": "UL", "cote": "24", "date": "12 (ex.)", ...}

Un CSV ne contient que du texte : ni nombres, ni True, ni listes.

Les pièges du CSV

Le séparateur

Il n'y a pas de norme unique : la virgule est l'usage international, mais un tableur réglé en français exporte avec un point-virgule.

Si on ne précise rien, Python suppose une virgule.

Notre fichier utilise le point-virgule, parce que les commentaires contiennent des virgules :

…;MS 249;13;1r-10v;Digitisation note: … from the old foliation, both numbers are provided.;…

Lu avec la virgule par défaut, il provoque une erreur à la ligne de Merton :

ParserError: Error tokenizing data. C error: Expected 1 fields in line 14, saw 2
→ sep=";"

La virgule décimale

Un tableur réglé en français écrit les nombres décimaux avec une virgule, alors que Python attend un point.

Si on ajoutait au corpus la hauteur des manuscrits en centimètres :

cote;hauteur_cm
24;30,5

Sans decimal=",", 30,2 est lu comme du texte :

- on ne peut pas calculer de moyenne ;
- le tri se fait dans l'ordre alphabétique, si bien que "100,5" passe avant "30,2".

→ decimal=","

A noter : on retrouvera ces paramètres avec read_csv()

Une question, deux réponses

La question : combien de manuscrits du corpus chaque ville conserve-t-elle ?

Avec ce qu'on connaît déjà

Si corpus est une liste de dictionnaires (séances 1 à 5) :

```
compte = {}
for manuscrit in corpus:
    ville = manuscrit["ville"]
    if ville in compte:
        compte[ville] += 1
    else:
        compte[ville] = 1
print(compte)
# {'Bern': 1, 'Città del Vaticano': 1, 'London': 4,
#  'Aberdeen': 1, 'Cambridge': 1, 'Oxford': 4, ...}
```

Il faut une boucle, une condition et un dictionnaire, et le résultat n'est pas trié.

Avec pandas

```
import pandas as pd
df = pd.read_csv("corpus.csv", sep=";")
print(df["ville"].value_counts())
# ville
# London              4
# Oxford              4
# Bern                1
# Città del Vaticano  1
# ...
```

Une ligne suffit, et le résultat est trié du plus fréquent au moins fréquent.

Deux conventions

- **pd** : on importe toujours pandas sous ce nom court.
- **df** (pour *DataFrame*) : c'est le nom habituel d'un tableau. On le trouve dans la documentation et dans presque tous les exemples en ligne.

DataFrame et Series

Deux structures

- **Le DataFrame** est un tableau complet, en deux dimensions (lignes et colonnes) : c'est la feuille du tableur.
- **La Series** est une seule colonne, en une dimension.

```
print(type(df))          # <class 'pandas.DataFrame'>
print(type(df["cote"]))  # <class 'pandas.Series'>

print(df["cote"])
# 0            318
# 1    Pal. lat. 1074
# 2       Stowe 1067
# ...
# Name: cote, dtype: str
```

À gauche, l'**index** numérote les lignes à partir de 0, comme les indices d'une liste.

Des objets, comme en POO

DataFrame et Series sont des classes : df est une instance de DataFrame. Comme nos objets, il possède :

- des **attributs**, qui décrivent le tableau, sans parenthèses :

```
df.shape    # (15, 9) : 15 lignes, 9 colonnes
```

- des **méthodes**, qui agissent sur le tableau, avec des parenthèses :

```
df.head()                # les 5 premières lignes
df["ville"].value_counts()  # compter les valeurs
```

Pour reprendre l'analogie de la langue : df est un nouveau nom, et ses méthodes sont de nouveaux verbes.

Lire un fichier : read_csv()

Lire le fichier du corpus

On construit le chemin avec Path :

```
from pathlib import Path
import pandas as pd

fichier = Path("donnees") / "corpus.csv"
df = pd.read_csv(fichier, sep=";")
```

read_csv() lit tout le fichier d'un coup. La première ligne devient le nom des colonnes, et chaque ligne suivante devient une ligne du tableau.

Rappel des pièges

```
pd.read_csv(fichier, sep=";")              # séparateur
pd.read_csv(fichier, sep=";", decimal=",")    # virgule décimale
```

Sans sep=";", pandas suppose une virgule et la lecture échoue.

Les cellules vides deviennent NaN

NaN (*Not a Number*) signale une valeur manquante :

```
        cote    version                 manifeste
3         24  [*B-Bes6]                       NaN
8  Ashmole 1511       NaN  https://iiif.bodleian.ox.ac…
```

Aberdeen n'a pas de manifeste, et Ashmole 1511 n'a pas de version : ce sont les ;; dans le fichier.

Explorer un tableau

Examiner la structure (attributs, sans parenthèses)

```
df.shape      # (15, 9) : 15 lignes, 9 colonnes
df.columns    # Index(['sigle', 'version', 'ville', 'bibliotheque',
              #        'cote', 'date', 'folios', 'commentaire',
              #        'manifeste'], dtype='str')
```

Faire le bilan des colonnes : info()

```
df.info()
#  #   Column       Non-Null Count  Dtype
#  0   sigle        15 non-null     str
#  …
# 5   date         15 non-null     str
# …
#  8   manifeste    14 non-null     str
```

Pour chaque colonne, info() donne le nombre de valeurs renseignées et le type.

Afficher un extrait (méthodes, avec parenthèses)

```
df.head()     # les 5 premières lignes
df.tail()     # les 5 dernières lignes
df.head(3)    # les 3 premières lignes
```

Résumer les valeurs : describe()

```
df.describe()
#        sigle  ...                    manifeste
# count     15  ...                           14
# unique     8  ...                           14
# …
# [4 rows x 9 columns]
```

Pour des colonnes de texte, describe() compte les valeurs, les valeurs distinctes et la plus fréquente. Pour des colonnes numériques, il donnerait la moyenne, le minimum et le maximum.

Sélectionner des colonnes et des lignes

Sélectionner des colonnes : []

- Une colonne, avec son nom entre crochets, donne une Series :

```
df["cote"]
```

- Plusieurs colonnes, avec une **liste** de noms (d'où les doubles crochets), donnent un DataFrame :

```
df[["cote", "ville"]]
  #           cote            ville
  # 0          318             Bern
  # 1  Pal. lat. 1074  Città del Vaticano
  # 2      Stowe 1067           London
  # ...
```

Sélectionner des lignes

- **loc sélectionne par étiquette**, c'est-à-dire par le nom de la ligne dans l'index :

```
df.loc[8]                  # la ligne d'Ashmole 1511
df.loc[8, "cote"]            # 'Ashmole 1511'
df.loc[0:2, ["cote", "ville"]]    # lignes 0 à 2, fin incluse
```

- **iloc sélectionne par position**, comme les indices d'une liste :

```
df.iloc[0]      # la première ligne
df.iloc[0:3]    # lignes 0 à 2, fin exclue
```

Pour l'instant, étiquettes et positions coïncident. Elles se séparent dès qu'on filtre ou qu'on trie le tableau : la ligne d'Ashmole garde alors l'étiquette 8, quelle que soit sa nouvelle position.

⚠ **Attention**

`df.loc["cote"]` cherche une **ligne** nommée « cote » et provoque une KeyError. Pour une colonne, on écrit `df["cote"]`.

Filtrer avec une condition

Une condition donne une Series de booléens

```
df["ville"] == "London"
# 0    False
# 1    False
# 2     True
# 3    False
# …
```

Placée entre crochets, cette Series ne garde que les lignes marquées True :

```
df[df["ville"] == "London"]
#                cote   ville      date
# 2         Stowe 1067  London  12 (inc.)
# 5         Add. 11283  London  12 (inc.)
# 6        Harley 4751  London   12 (ex.)
# 11  Cotton MS Nero A V  London        12
```

Les lignes gardent leur étiquette (2, 5, 6, 11).

Filtrer et choisir les colonnes : loc

```
df.loc[df["ville"] == "London", ["cote", "date"]]
```

Combiner des conditions

- **&** signifie « et » : les deux conditions doivent être vraies.

```
# extrait
df.loc[(df["ville"] == "Oxford") & (df["sigle"] == "2F"), ["cote"]]
 # 7     Bodl. 533
 # 8  Ashmole 1511
 # 9           61
```

- **|** signifie « ou » : une des deux conditions suffit.

```
df.loc[(df["ville"] == "Bern") | (df["ville"] == "Leipzig"), ["cote"]]
```

⚠ **Attention**

- Avec pandas, on n'utilise pas les mots and et or de la séance 1, mais les symboles & et |.
- Les parenthèses autour de chaque condition sont obligatoires. Sans elles, on obtient :

```
TypeError: unsupported operand type(s) for &: 'str' and 'StringArray'
```

Les valeurs manquantes

Repérer : isna()

isna() renvoie True pour chaque cellule vide (NaN).

```
df.isna().sum()      # nombre de cellules vides par colonne
# version        6
# commentaire    11
# manifeste       1
# ...
```

Comme toute Series de booléens, isna() sert aussi à filtrer :

```
df.loc[df["manifeste"].isna(), ["cote", "ville"]]
#   cote     ville
# 3   24  Aberdeen
```

Supprimer : dropna()

```
df.dropna()                    # 3 lignes sur 15 !
df.dropna(subset=["manifeste"])    # 14 lignes
```

⚠ **Attention :** sans précision, dropna() supprime toute ligne qui contient au moins une cellule vide. Ici, il ne reste que 3 manuscrits. Avec subset, on choisit la colonne à vérifier.

Remplacer : fillna()

```
df["commentaire"] = df["commentaire"].fillna("aucun")
```

On remplit **colonne par colonne**, avec une valeur du bon type. Ex : du texte dans une colonne de texte.

⚠ **Attention :** dropna() et fillna() ne modifient pas df, ils renvoient un nouveau tableau.

Créer une nouvelle colonne

Le problème : date n'est pas un nombre

La slide 12 l'avait montré : date est de type str. Pour la convertir en nombre, on utilise astype() :

```
df["date"].astype(int)
# ValueError: invalid literal for int() with base 10: '10-11'
```

La conversion échoue, parce que les dates notent des siècles approximatifs : 9, 10-11, 12 (ex.).

La solution : extraire le siècle de départ

Dans toutes nos dates, les deux premiers caractères donnent le siècle. .str applique une opération de texte à chaque cellule, ici une découpe, comme titre[:2] en séance 1 :

```
df["date"].str[:2]
# 0    9
# 1   10
# 2   12
# ...
```

Créer la colonne siecle

On convertit le résultat en nombre, puis on le range dans une nouvelle colonne, comme on ajouterait une clé à un dictionnaire :

```
df["siecle"] = df["date"].str[:2].astype(int)
#            cote      date  siecle
# 0           318         9       9
# 1  Pal. lat. 1074      10-11      10
# 2      Stowe 1067  12 (inc.)      12
# 3            24   12 (ex.)      12
```

On peut maintenant filtrer sur un nombre

```
df.loc[df["siecle"] <= 10, ["cote", "date"]]
#            cote   date
# 0           318      9
# 1  Pal. lat. 1074  10-11
```

⚠ **Limite :** cette découpe ne marche que sur notre corpus. Une date notée 8-9 donnerait 8-, et astype(int) échouerait.

Trier et compter

Trier : sort_values()

```
df.sort_values("siecle")                  # du plus ancien au plus récent
df.sort_values("siecle", ascending=False)    # du plus récent au plus ancien
#          cote  siecle
# 4          53     14
# 13  Cod. gr. 35     14
# 7     Bodl. 533     13
# …
```

Les lignes gardent leur étiquette (4, 13, 7) : l'index ne se renumérote pas après un tri.

Compter : value_counts()

On l'a vu à la slide 9 avec les villes. La méthode s'applique à n'importe quelle colonne :

```
df["siecle"].value_counts()
# siecle
# 12    7
# 13    4
# …
```

Le XIIe siècle domine le corpus.

```
df["sigle"].value_counts()
# sigle
# 2F    7
# PT    2
# ...
```

La moitié des manuscrits appartiennent à la Seconde Famille (2F).

⚠ **Attention :** comme `dropna()`, `sort_values()` ne modifie pas `df`. Pour garder le tri, il faut écrire `df = df.sort_values("siecle")`.

Écrire un fichier : to_csv()

Enregistrer une sélection

On sélectionne les manuscrits du XIIe siècle, puis on les enregistre dans un nouveau fichier :

```
selection = df.loc[df["siecle"] == 12, ["cote", "ville", "siecle"]]
fichier = Path("donnees") / "xiie_siecle.csv"
selection.to_csv(fichier, sep=";", index=False)
```

On retrouve les paramètres de la lecture : sep=";" pour le séparateur. Le fichier est enregistré en UTF-8 par défaut.
Le fichier obtenu, ouvert dans un éditeur de texte :

```
cote;ville;siecle
Stowe 1067;London;12
24;Aberdeen;12
Add. 11283;London;12
…
```

Pourquoi index=False ?

Sans ce paramètre, pandas écrit aussi l'index dans une première colonne sans nom :

```
;cote;ville;siecle
2;Stowe 1067;London;12
3;24;Aberdeen;12
…
```

Ces numéros n'ont aucun sens hors de Python. À la relecture, ils deviendraient une colonne de plus.

La boucle est bouclée

Le fichier peut être rouvert avec read_csv(), ouvert dans un tableur ou envoyé à un collègue qui ne code pas.

Notions à retenir

Le CSV

- Un CSV est un fichier texte : une ligne d'en-tête, puis une notice par ligne.
- Le séparateur varie : la virgule par défaut, souvent le point-virgule en français.
- Un CSV ne contient que du texte. Une cellule vide (;;) devient NaN.

pandas

- import pandas as pd, et df pour le tableau : deux conventions.
- Un DataFrame est un tableau, une Series une colonne. Ce sont des objets, avec des attributs (shape, columns) et des méthodes (head(), info()).

Lire et explorer

- pd.read_csv(fichier, sep=";"), avec decimal="," si besoin.
- head(), tail(), info(), describe(), shape, columns.

Sélectionner et filtrer

- df["cote"] pour une colonne, df[["cote", "ville"]] pour plusieurs.
- loc par étiquette, iloc par position.
- df.loc[condition, colonnes], en combinant les conditions avec & et |, chacune entre parenthèses.

Nettoyer

- isna() pour repérer, dropna(subset=...) pour supprimer, fillna() pour remplacer, colonne par colonne.
- astype() pour convertir, .str pour travailler sur le texte de chaque cellule.
- df["siecle"] = ... pour créer une colonne.
- sort_values() pour trier, value_counts() pour compter.

Écrire

- to_csv(fichier, sep=";", index=False).

⚠ **Attention :** dropna(), fillna() et sort_values() renvoient un nouveau tableau. Pour garder le résultat, il faut l'affecter.